

Your Markdown document is ready [file-tag: code-generated-file-0-1785011100511797779]

Conteúdo do Documento de Resumo

Documento de Handoff & Resumo de Contexto - Projeto Estação de Sensores ESP32

****Status do Projeto:**** Funcional (Comunicação ESP-NOW + Servidor Web Assíncrono com LittleFS + HTML/CSS/JS Decouplados)

1. Visão Geral e Objetivo

Desenvolvimento de um sistema de monitoramento de sensores distribuído utilizando placas ESP32. O sistema consiste em módulos ****Emissores**** (que leem sensores como DHT11 e transmitem via ESP-NOW) e um módulo ****Receptor/Central**** (que recebe os dados via rádio, exibe no display local OLED, hospeda uma página Web local no LittleFS e fornece uma API JSON em tempo real via Wi-Fi para smartphones/navegadores).

1.1 Objetivo

Chegar a um protótipo tipo maquete com um sistema aplicável em que poderemos acionar um secador de cabelos via site hospedado em host, causando elevação de temperatura, e forçando o sistema de exaustão (ventilador) trazendo a temperatura de volta ao normal, com acionamento e desligamento automático.

2. Hardware e Infraestrutura

- ****Placas:**** ESP32 LoRa LyliGo (TTGO T-Display / DevKit v1) - 2 unidades
 - interface para ESP32 LoRa REV2.0/2022 com fonte de alimentação integrada 127vac~240vav-5vdc 0.6A - 2 unidades
- ****Mapeamento de Portas COM (Hub USB com Fonte Externa 5v 3A):****
 - ****COM3:**** Placa Receptora (Central Web)
 - ****COM4:**** Placa Emissora (Sensor DHT11)
- ****Periféricos & Sensores:****
 - Display OLED I2C (SSD1306): Pinos SDA=21, SCL=22 - cada unidade ESP32 carrega um Oled
 - Sensor DHT11 (na placa emissora) 1 unidade
 - Relé de estado sólido FOTEK SSR-25DA 25A 3-32vdc/24-380vac com dissipadores/fixadores de trilho DIN TS35 - 2 unidades
 - placa de relé HW-316 com 4 relés independentes JQC3F-05vdc-c 250vac 10A
 - 4 mini protoboards
 - 1 secador de cabelos residencial de 3500W 127v
 - 1 ventilador residencial de 50W 127v
- ****Rede / Wi-Fi:****
 - Modo Wi-Fi do Receptor: `WIFI\_AP\_STA` (Canal 1 fixo)
 - SSID da Rede Local: `ESP32\_Estacao` | Senha: `12345678`
 - IP de Acesso: `192.168.4.1`
 - **Recursos extras disponíveis no laboratório****
 - Multímetro simples de uso geral para medições de componentes, continuidade e Vac Vdc
 - Roteador Wi-Fi residencial `NILSON 2.4`

- Senha `81111270in`
- Assinatura Hostgator M anual

3. Arquitetura de Software e Decisões Técnicas

- 1. **Ambiente de Desenvolvimento:**** VS Code + PlatformIO.
- 2. **Separação de Ambientes (`platformio.ini`):****
 - Uso da diretiva `build\_src\_filter` para compilar apenas o arquivo relevante em cada placa sem necessidade de renomear extensões.
 - Isolamento de dependências de bibliotecas (`lib\_deps`) por ambiente (Emissor não carrega servidor Web).
- 3. **Sistema de Arquivos (`LittleFS`):****
 - Arquivos web armazenados na pasta `data/` do projeto e gravados na Flash do ESP32 via target `uploadfs`.
 - Estrutura Web totalmente modularizada e desacoplada:
 - `data/index.html` (Estrutura da página)
 - `data/style.css` (Estilização)
 - `data/script.js` (Lógica AJAX com `fetch()` assíncrono a cada 2s com cache-busting)
- 4. **Comunicação e Servidor:****
 - ****Emissor -> Receptor:**** ESP-NOW (envio de struct de dados com temperatura, umidade e contador de pacotes).
 - ****Receptor -> Cliente Web:**** Servidor assíncrono (`ESPAsyncWebServer`) servindo arquivos estáticos e endpoint `/dados` fornecendo JSON.

4. Estrutura de Pastas do Projeto

```
MEU_PROJETO/
├── platformio.ini # Configuração dos ambientes emissor e receptor
├── data/ # Arquivos do Servidor Web (LittleFS - Receptor)
│   ├── index.html # Estrutura HTML
│   ├── style.css # Estilo visual
│   └── script.js # Atualização dinâmica (AJAX fetch)
├── src/
│   ├── emissor.cpp # Código Leve: Leitura do DHT11 + Envio ESP-NOW
│   └── receptor.cpp # Código Central: ESP-NOW + OLED + WebServer + LittleFS
```

5. Arquivos de Configuração e Código Atual

5.1. `platformio.ini`

```
[env]
platform = espressif32
board = esp32dev
framework = arduino
monitor_speed = 115200
board_build.filesystem = littlefs
```

```
# CONFIGURAÇÃO DA PLACA EMISSORA

[env:ttgo_emissor]
upload_port = COM4
monitor_port = COM4
build_src_filter = +<\*> -<receptor.cpp> -<main.cpp>
lib_deps =
adafruit/DHT sensor Library
adafruit/Adafruit Unified Sensor
adafruit/Adafruit GFX Library
adafruit/Adafruit SSD1306

# CONFIGURAÇÃO DA PLACA RECEPTORA

[env:ttgo_receptor]
upload_port = COM3
monitor_port = COM3
build_src_filter = +<\*> -<emissor.cpp> -<main.cpp>
lib_deps =
adafruit/Adafruit GFX Library
adafruit/Adafruit SSD1306
https://github.com/me-no-dev/ESPAsyncWebServer.git
https://github.com/me-no-dev/AsyncTCP.git

### 5.2. `src/receptor.cpp`

#include <esp_now.h>
#include <WiFi.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <Arduino.h>
#include <ESPAsyncWebServer.h>
#include <LittleFS.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

AsyncWebServer server(80);

uint8_t macDestino[] = {0xA0, 0xDD, 0x6C, 0x75, 0x0E, 0x14};

typedef struct struct_mensagem
{
int contador;
char status[10];
float temperatura;
float umidade;
} struct_mensagem;

struct_mensagem dadosRecebidos;
struct_mensagem dadosResposta;
esp_now_peer_info_t peerInfo;
```

```
void AoReceber(const uint8_t *mac_addr, const uint8_t *incomingData, int len)
{
    memcpy(&dadosRecebidos, incomingData, sizeof(dadosRecebidos));

    display.clearDisplay();
    display.setCursor(0, 0);
    display.println("PLACA B (RECEPTOR)");
    display.setCursor(0, 16);
    display.print("Temp: ");
    display.print(dadosRecebidos.temperatura, 1);
    display.println(" C");
    display.setCursor(0, 32);
    display.print("Umid: ");
    display.print(dadosRecebidos.umidade, 1);
    display.println(" %");
    display.setCursor(0, 48);
    display.print("Pacote: #");
    display.println(dadosRecebidos.contador);
    display.display();

    dadosResposta.contador = dadosRecebidos.contador;
    strcpy(dadosResposta.status, "ACK OK");
    esp_now_send(macDestino, (uint8_t *)&dadosResposta, sizeof(dadosResposta));
}

void setup()
{
    Serial.begin(115200);
    delay(1000);

    Wire.begin(21, 22);
    display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
    display.setTextColor(WHITE);
    display.setTextSize(1);

    //inicializa LittleFS com opção de formatar em caso de erro na 1ª tentativa
    if (!LittleFS.begin(true)) {
        Serial.println("Erro ao montar o LittleFS!");
    }

    //Inicializa WiFi
    WiFi.mode(WIFI_AP_STA);
    WiFi.softAP("ESP32_Estacao", "12345678", 1);

    //inicializa ESP-NOW
    if (esp_now_init() == ESP_OK) {
        esp_now_register_recv_cb(AoReceber);
        memcpy(peerInfo.peer_addr, macDestino, 6);
        peerInfo.channel = 1;
        peerInfo.encrypt = false;
        esp_now_add_peer(&peerInfo);
    }
```

```
//Configuração das rotas do servidor

//servir index.html
server.on("/", HTTP_GET, [](AsyncWebServerRequest *request){
request->send(LittleFS, "/index.html", "text/html");
});

//servir style.css
server.on("/style.css", HTTP_GET, [](AsyncWebServerRequest *request){
request->send(LittleFS, "/style.css", "text/css");
});

//servir script.js
server.on("/script.js", HTTP_GET, [](AsyncWebServerRequest *request){
request->send(LittleFS, "/script.js", "text/javascript");
});

//servir json
server.on("/dados", HTTP_GET, [](AsyncWebServerRequest *request){
String json = "{";
json += "\"temperatura\":\"" + String(dadosRecebidos.temperatura, 1) + ",";
json += "\"umidade\":\"" + String(dadosRecebidos.umidade, 1) + ",";
json += "\"contador\":\"" + String(dadosRecebidos.contador);
json += "\"}";
request->send(200, "application/json", json);
});

// inicia servidor Web
server.begin();
}

void loop()
{
}

### 5.3. Arquivos da Pasta `data/`

#### `data/index.html`

<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Painel ESP32</title>
  <link rel="stylesheet" type="text/css" href="style.css">
  <script src="script.js" defer</script>
</head>
<body>
  <h1>Estação de Sensores</h1>

  <div class="card">
    <p>Temperatura</p>
    <div class="valor"><span id="temp">--</span> °C</div>
```

```
</div>

<div class="card">
  <p>Umidade</p>
  <div class="valor"><span id="umid">--</span> %</div>
</div>

<p>Pacotes recebidos: <b id="contador">0</b></p>
</body>
</html>

#### `data/style.css`

body {
font-family: Arial, sans-serif;
text-align: center;
margin-top: 50px;
background-color: #f4f4f9;
}

h1 {
color: #333;
}

.card {
background: white;
padding: 20px;
margin: 10px auto;
width: 250px;
border-radius: 10px;
box-shadow: 0 2px 5px rgba(0,0,0,0.2);
}

.valor {
font-size: 2em;
color: #007bff;
font-weight: bold;
}

#### `data/script.js`

function atualizarDados() {
fetch('/dados?t=' + new Date().getTime())
.then(response => response.json())
.then(data => {
document.getElementById('temp').innerText = data.temperatura;
document.getElementById('umid').innerText = data.umidade;
document.getElementById('contador').innerText = data.contador;
})
.catch(err => console.log('Erro ao buscar dados:', err));
}

setInterval(atualizarDados, 2000);
```

6. Fluxo de Comandos e Gravação (PlatformIO CLI)

1. ****Upload do LittleFS (Receptor / COM3):****
`pio run -e ttgo\_receptor -t uploadfs`
2. ****Upload do Firmware do Receptor (COM3):****
`pio run -e ttgo\_receptor -t upload`
3. ****Upload do Firmware do Emissor (COM4):****
`pio run -e ttgo\_emissor -t upload`

7. Próximos Passos

- inserção de botão no HTML para acionamento de aquecedor (secador de cabelos por 30seg)
- inserção de código para acionamento de ventilador, caso a temperatura ultrapasse um limite determinado, e desligue quando chegar à temp determinada
- criação de um banco de dados sql no provedor hostgator
- inserção de dados automaticamente (hora/temperatura/umidade) no banco de dados com PHP
- inserção de site para acesso global na hostgator tão simples quanto o site do LittleFS
- inserção de logo e relatorio de dados do banco sql no site
- teste de campo para verificação de funcionamento à distancia com powerbank alimentando o receptor, e teste via internet com smartfone
- configurar nova unidade esp32 LyLigo oled (que ainda será adquirida) para servir de gateway e aumentar alcance
- definir todas as necessidades de aplicações com seus respectivos sensores (apenas quando todos os passos anteriores forem consolidados)
- Refatorar o código C++ aplicando ****Programação Orientada a Objetos (POO)**** com classes dedicadas (`DisplayManager`, `WebServerManager`, `SensorManager`).
- Suporte a múltiplos transmissores.

8. Informações gerais de parametros para IA auxiliar

- respostas diretas e sem abstrações.
- cada fase, e cada passo deve ser respeitado, só passamos adiante depois de confirmação de funcionamento, e commit local e nuvem.

8.1 Lista de commits com descrições e respectivas datas

- Date: Sat Jul 25 17:39:53 2026 Adição de documento resumo projeto
- Date: Sat Jul 25 17:13:46 2026 adiciona serv web assincrono com LittleFs (html,css,js)
- Date: Wed Jul 22 00:48:32 2026 inserção DHT11 sensor temperatura/umidade
- Date: Tue Jul 21 15:17:37 2026 Comunic Bidirecional base ESP-NOW OLED